

Relational Database Management Systems

Data Science for Economics

Damian Clarke

dcc213@exeter.ac.uk

Department of Economics, University of Exeter

5 March, 2026

Why Databases?

The Problem with Files

Suppose you are managing data for a university. You might have:

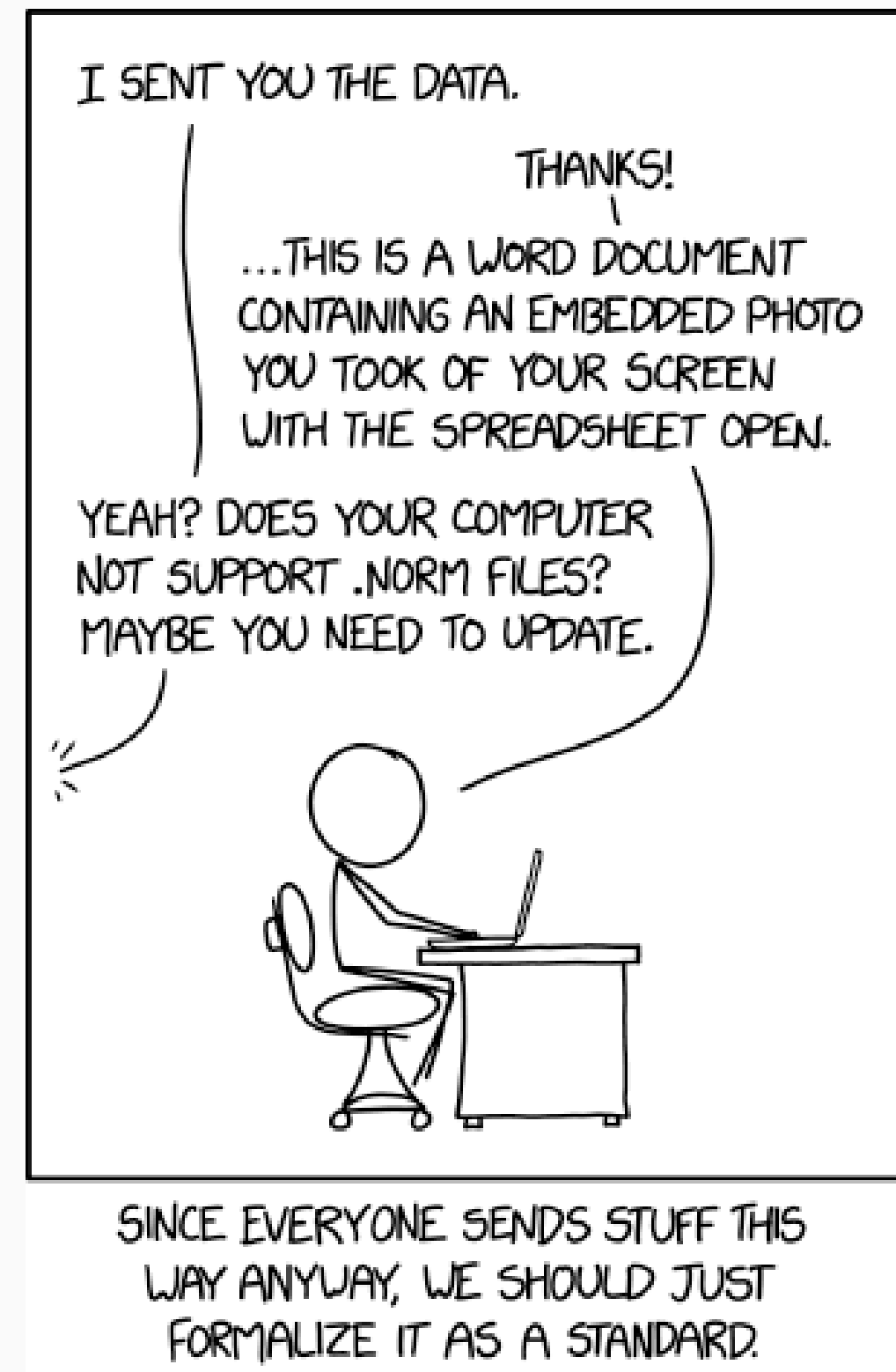
- A spreadsheet of students and their grades
- A spreadsheet of courses and lecturers
- A spreadsheet of student enrolments

This is a **file processing system** — and it starts to break down fast.

Think about:

1. A **library** managing millions of books and members
2. A **company** tracking employees, sales, inventory
3. A **university** managing students, modules, grades, staff

What could go wrong?



Source: xkcd.com/2116

File Processing Systems: The Problems

A file processing system stores data categorically with all required data for particular uses in individual files.

It has some advantages: it is cheap and easy to use. But it has serious disadvantages:

- **Data redundancy** — the same data stored in multiple places
- **Data inconsistency** — copies get out of sync
- **Data isolation** — data in separate files hard to combine
- **Program dependence** — data format tied to specific programs
- **Atomicity issues** — partial updates leave data corrupted
- **Data security** — hard to control who sees what

Database Management Systems: The Solution

A **DBMS** = Database + Management System

- **Database**: a structured, organised collection of related data
- **Management System**: the software that stores, retrieves, updates, and secures that data

Advantages over flat files:

- No redundant data — a fact is stored once, in one place
- Data consistency and integrity — rules are enforced automatically
- Data security and privacy — fine-grained access control
- Easy, fast access — the DBMS does the heavy lifting
- Easy recovery from failures
- Flexible and scalable — add tables, users, and data without restructuring

What Exactly is a Relational Database?

A **relational database** organises data into **tables** (also called *relations*).

Each table has:

- **Columns** (attributes): the fields, e.g. `Lec_name`, `Lec_dept`
- **Rows** (tuples): the individual records, e.g. one lecturer

Tables are linked to each other through **keys**: shared columns that create explicit relationships.

This is the “relational” in relational database: the data model is built on the mathematical concept of a *relation* (from set theory).

RDBMS = Relational Database Management System: the software that implements this model.

Example: two related tables

Lec_ID	Lec_name	Lec_dept
L211	Abdul	Comp Sci
L555	Sarah	Economics

Mod_code	Mod_name	Lec_ID
C01	Intro to CS	L211
E15	Econometrics	L555

`Lec_ID` is the link between them.

What is SQL?

SQL (Structured Query Language) is the language we use to *talk to* a relational database.

It was originally called **Structured English Query Language** (SEQUEL), and the name reflects its design: SQL reads like natural English.

```
SELECT Lec_name, Lec_dept  
FROM   Lecturer  
WHERE  Lec_dept = 'Economics';
```

Read it out loud: “*Select the name and department from the Lecturer table where the department is Economics.*”

SQL is:

- **Declarative**: you say *what* you want, not *how* to get it
- **Standardised**: the core language works across PostgreSQL, MySQL, SQLite, SQL Server, and more
- **Universal**: more or less every data scientist, analyst, and data engineer uses it

SQL and RDBMS: How They Fit Together

Think of the **RDBMS** as the system, and **SQL** as the way you access the system.

- The RDBMS stores and manages data (SQLite, PostgreSQL, MySQL...)
- SQL is the language you use to drive it
- You type SQL; the RDBMS figures out how to execute it efficiently

SQL covers four main areas:

Area	Abbreviation	Examples
Define structure	DDL	CREATE TABLE, DROP
Query data	DQL	SELECT
Modify data	DML	INSERT, UPDATE, DELETE
Control access	DCL	GRANT, REVOKE

This lecture's roadmap:

1. RDBMS concepts
2. SQLite3 at the command line
3. DDL: defining tables
4. DQL: querying data
5. DML: modifying data
6. JOINS: combining tables
7. Remote connections
8. Bringing it into Python

The Math Behind It All

There is substantial theory behind DBMS... more than you might expect.

This is captured in **Data Models**, which formalise what data are and how they relate to each other.

At a **conceptual** level, we describe data using an **Entity-Relationship (E-R) Diagram**.

At a **practical** level, this leads to the RDBMS model we use today, first described formally by **E. F. Codd** in a landmark 1970 IBM paper.

We will encounter a piece of this mathematics: **relational algebra**, when we discuss JOINS.

If you want to explore E-R diagrams further, see:

[geeksforgeeks.org/introduction-of-er-model/](https://www.geeksforgeeks.org/introduction-of-er-model/)

Edgar F. Codd (1923-2003)

- Born in Portland, Dorset, UK
- Worked at IBM's San Jose Research Lab
- 1970 paper: *"A Relational Model of Data for Large Shared Data Banks"*
- Won the Turing Award in 1981

Everything we do with SQL ultimately traces back to his work.

[Wikipedia: Edgar F. Codd](#)

Principles of RDBMS

RDBMS: The Building Blocks

A Relational Database Management System consists of:

Concept	What it means
Table (relation)	A 2D structure of rows and columns
Record (tuple)	One row in a table
Field (attribute)	One column: a named data item
Domain	The set of allowed values for a field
Schema	The structure/blueprint of the database
Instance	The actual data in the database at a point in time
Keys	Special fields used to identify and link records

Keys: The Glue of Relational Databases

Keys are fundamental to how tables relate to each other:

- **Super key:** any set of attributes that uniquely identifies a row
- **Candidate key:** a minimal super key (no redundant attributes)
- **Primary key:** the chosen candidate key for a table – *must be unique and NOT NULL*
- **Foreign key:** a field in one table that references the primary key of another

```
CREATE TABLE Lecturer (  
  Lec_ID   TEXT NOT NULL PRIMARY KEY,  
  Lec_name TEXT,  
  Lec_dept TEXT  
);  
CREATE TABLE Module (  
  Mod_code TEXT NOT NULL PRIMARY KEY,  
  Mod_name TEXT,  
  Lec_ID   TEXT,  
  FOREIGN KEY (Lec_ID) REFERENCES Lecturer(Lec_ID)  
);
```

Constraints

Constraints enforce the rules of your data:

Constraint	Meaning
NOT NULL	Field cannot be empty
UNIQUE	No two rows can have the same value
PRIMARY KEY	Unique + Not null: identifies each row
FOREIGN KEY	Must match a value in another table (or be NULL)
CHECK	Value must satisfy a condition
DEFAULT	Provides a default value when none is given

These make your database **self-enforcing**. Bad data cannot be inserted.

Cardinality: How Tables Relate

Cardinality describes the numerical relationship between records in two tables:

- **One-to-one**: One lecturer has one office; one office has one lecturer
- **One-to-many**: One lecturer teaches many modules; each module has one lecturer
- **Many-to-many**: Students enrol in many modules; modules have many students

In our teaching database:

- `Lecturer` → `Module` is **one-to-many** (one lecturer, many modules)
- A student-module enrolment would be **many-to-many**

A Note on Normalisation

Why do we have multiple tables instead of one big table?

This is the idea behind **normalisation**: organising data to reduce redundancy and improve integrity.

Informally:

- Don't store the same fact in more than one place
- Each table should describe one thing
- Dependencies should be explicit via keys

For example: if Abdul's department changes, we update it in **one row** of `Lecturer`, not in every row of `Module`.

See <https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/> for further information.

Installing SQLite3 and the CLI

What is SQLite3?

SQLite is a **lightweight, serverless** relational database engine.

- No separate database server to install or run
- The entire database lives in a **single file** on disk
- Built into Python: no extra install needed
- Powers many common applications: Firefox, Android, iOS, WhatsApp, and more

For learning SQL and for many real data science tasks, SQLite3 is perfect:

```
"SQLite is the most widely deployed database engine in the world"  
-- SQLite documentation
```

We will start by working directly from the **command line** – the same way this is often done in practice.

Installation

macOS

SQLite3 is pre-installed. Check with:

```
sqlite3 --version
```

If missing, install via Homebrew:

```
brew install sqlite3
```

Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install sqlite3
```

Windows

Download the precompiled binary from sqlite.org/download.html and add the folder to your system PATH.

The recommended approach for this course is **WSL** (Windows Subsystem for Linux):

```
# In a WSL terminal  
sudo apt install sqlite3  
sqlite3 --version
```

Depending on how you were using the CLI previously, you may wish to alter the above...

The SQLite3 Shell: Getting In

Open a terminal and launch SQLite3 with a database filename:

```
# Creates teaching.db if it doesn't exist, opens it if it does
sqlite3 teaching.db
```

You are now inside the interactive shell. The prompt changes to:

```
SQLite version 3.x.x
Enter ".help" for usage hints.
sqlite>
```

To exit at any time:

```
sqlite> .quit
```

Note: SQLite3 only creates the `.db` file once data is written to it. Simply opening is not enough.

Essential Dot Commands

The SQLite3 shell has special “dot commands” that control the session – not SQL, but very useful for us:

```
.help          -- list all dot commands
.tables        -- list all tables in the database
.schema        -- show CREATE statements for all tables
.schema Lecturer -- show CREATE for one specific table
.headers on    -- show column names in query output
.mode column   -- align output in neat columns
.mode table    -- pretty-print with box borders
.width 20 20 15 -- set column widths manually
.read script.sql -- run a SQL script file
.output file.txt -- redirect query output to a file
.quit         -- exit the shell
```

Worth running `.headers on` **and** `.mode column` **at the start of a session** – the raw pipe-separated output can be hard to read.

Formatting Output Nicely

Without formatting:

```
L211|Abdul|Computer Science
L555|Sarah|Economics
```

With `.headers on` and `.mode column`:

```
sqlite3 teaching.db
```

```
sqlite> .headers on
sqlite> .mode column
```

```
sqlite> SELECT * FROM Lecturer;
Lec_ID  Lec_name  Lec_dept
-----  -
L211    Abdul    Computer Science
L555    Sarah    Economics
L179    Amy      Physics
L234    Lou      Psychology
R101    Marco    Social Science
```

You can also try `.mode table` for a box-style layout.

Our First Database at the CLI

Running a SQL Script

Rather than typing every statement interactively, we write a `.sql` script and run it.

Save as `createTeachingTables.sql`:

```
DROP TABLE IF EXISTS Module;
DROP TABLE IF EXISTS Lecturer;

CREATE TABLE Lecturer (
  Lec_ID   TEXT NOT NULL PRIMARY KEY,
  Lec_name TEXT,
  Lec_dept TEXT
);
CREATE TABLE Module (
  Mod_code TEXT NOT NULL PRIMARY KEY,
  Mod_name TEXT,
  Lec_ID   TEXT,
  FOREIGN KEY (Lec_ID) REFERENCES Lecturer(Lec_ID)
);

INSERT INTO Lecturer VALUES ('L211','Abdul','Computer Science');
INSERT INTO Lecturer VALUES ('L555','Sarah','Economics');
INSERT INTO Lecturer VALUES ('L179','Amy','Physics');
INSERT INTO Lecturer VALUES ('L234','Lou','Psychology');

INSERT INTO Module VALUES ('C01','Intro to CS','L211');
INSERT INTO Module VALUES ('C02','DBMS','L211');
INSERT INTO Module VALUES ('C22','Data Science','L179');
INSERT INTO Module VALUES ('C39','Computer Networks','L179');
INSERT INTO Module VALUES ('E15','Econometrics','L555');
INSERT INTO Module VALUES ('U01','Dissertation',NULL);

INSERT INTO Lecturer VALUES ('R101','Marco','Social Science');
```

Running the Script

From the terminal: two equivalent ways:

```
# Option 1: pipe the script into sqlite3
sqlite3 teaching.db < createTeachingTables.sql

# Option 2: open the shell, then read the file
sqlite3 teaching.db
```

```
sqlite> .read createTeachingTables.sql
```

Verify it worked:

```
sqlite> .tables
Lecturer  Module
```

```
sqlite> .headers on
sqlite> .mode column
sqlite> SELECT * FROM Lecturer;
Lec_ID  Lec_name  Lec_dept
-----  -
L211    Abdul     Computer Science
...
```

What the Data Looks Like

```
sqlite> SELECT * FROM Module;
Mod_code  Mod_name          Lec_ID
-----  -
C01       Intro to CS       L211
C02       DBMS              L211
C22       Data Science      L179
C39       Computer Networks L179
E15       Econometrics      L555
U01       Dissertation
```

Notice:

- `U01` (Dissertation) has no `Lec_ID`: it is **NULL** (not a space, nor zero)
- `Marco` (`R101`) is in the `Lecturer` table but has no modules yet

These are the two interesting cases we will come back to when we discuss JOINS.

What We Have Built So Far

Two tables, linked by `Lec_ID`:

Lecturer (5 rows)

Lec_ID	Lec_name	Lec_dept
L211	Abdul	Computer Science
L555	Sarah	Economics
L179	Amy	Physics
L234	Lou	Psychology
R101	Marco	Social Science

Module (6 rows)

Mod_code	Mod_name	Lec_ID
C01	Intro to CS	L211
C02	DBMS	L211
C22	Data Science	L179
C39	Computer Networks	L179
E15	Econometrics	L555
U01	Dissertation	<i>NULL</i>

Two lecturers have no modules (Lou, Marco).
One module has no lecturer (U01). These edge cases matter a lot for JOINS.

Checking the Schema

```
sqlite> .schema
```

```
CREATE TABLE Lecturer (  
  Lec_ID    TEXT NOT NULL PRIMARY KEY,  
  Lec_name TEXT,  
  Lec_dept TEXT  
);  
CREATE TABLE Module (  
  Mod_code TEXT NOT NULL PRIMARY KEY,  
  Mod_name TEXT,  
  Lec_ID    TEXT,  
  FOREIGN KEY (Lec_ID) REFERENCES Lecturer(Lec_ID)  
);
```

This shows exactly how the tables were defined: *very* useful when you inherit a database you did not build yourself.

Query Languages and SQL

SQL: Structured Query Language

SQL (pronounced “S-Q-L” or “sequel”) was originally **Structured English Query Language**.

The name emphasises the natural, English-like syntax: reading a SQL query should feel like reading a sentence.

SQL includes four main sub-languages:

Sub-language	Abbreviation	What it does
Data Definition Language	DDL	CREATE, DROP, ALTER tables
Data Query Language	DQL	SELECT data
Data Manipulation Language	DML	INSERT, UPDATE, DELETE rows
Data Control Language	DCL	GRANT, REVOKE permissions

Today we focus on **DDL**, **DQL**, and **DML**.

SQL Syntax: The Anatomy of a Query

A SQL query has a predictable structure:

```
SELECT  column1, column2, ...      -- what to retrieve
FROM    table_name                -- where to look
WHERE    condition                 -- filter rows
GROUP BY column                  -- aggregate groups
HAVING   condition                -- filter groups
ORDER BY column ASC|DESC         -- sort results
LIMIT   n;                       -- restrict number of rows
```

Not all clauses are required — `SELECT ... FROM ...` is the minimum.

The best way to understand SQL is to **use it!**

SELECT: Querying Data

Basic SELECT

```
-- All columns from Lecturer
```

```
SELECT * FROM Lecturer;
```

Lec_ID	Lec_name	Lec_dept
L211	Abdul	Computer Science
L555	Sarah	Economics
L179	Amy	Physics
L234	Lou	Psychology
R101	Marco	Social Science

```
-- Specific columns only
```

```
SELECT Lec_name, Lec_dept FROM Lecturer;
```

Lec_name	Lec_dept
Abdul	Computer Science
Sarah	Economics
...	

Aliases and Calculated Fields

```
-- Rename columns in output with AS
```

```
SELECT Lec_name AS Name,  
       Lec_dept AS Department  
FROM   Lecturer;
```

Name	Department
Abdul	Computer Science
Sarah	Economics
Amy	Physics
Lou	Psychology
Marco	Social Science

Aliases make output cleaner and are essential in JOINS where column names from different tables can clash.

WHERE: Filtering Rows

```
-- Lecturers in Computer Science
SELECT *
FROM   Lecturer
WHERE  Lec_dept = 'Computer Science';
```

Lec_ID	Lec_name	Lec_dept
-----	-----	-----
L211	Abdul	Computer Science

```
-- Modules taught by L211
SELECT *
FROM   Module
WHERE  Lec_ID = 'L211';
```

Mod_code	Mod_name	Lec_ID
-----	-----	-----
C01	Intro to CS	L211
C02	DBMS	L211

WHERE with Multiple Conditions

-- AND: both conditions must be true

```
SELECT *
FROM   Lecturer
WHERE  Lec_dept ≠ 'Physics'
AND    Lec_ID   ≠ 'R101';
```

-- OR: either condition true

```
SELECT *
FROM   Module
WHERE  Lec_ID = 'L211' OR Lec_ID = 'L179';
```

-- IS NULL: find unassigned modules

```
SELECT *
FROM   Module
WHERE  Lec_ID IS NULL;
```

Mod_code	Mod_name	Lec_ID
-----	-----	-----
U01	Dissertation	

WHERE Lec_ID = NULL does **not** work. Always use IS NULL.

ORDER BY, LIMIT, and LIKE

```
-- Sort alphabetically by name
SELECT  Lec_name, Lec_dept
FROM    Lecturer
ORDER BY Lec_name ASC;
```

```
-- First 3 modules by code
SELECT  *
FROM    Module
ORDER BY Mod_code
LIMIT   3;
```

```
-- % is a wildcard meaning "any characters"
SELECT * FROM Module
WHERE  Mod_name LIKE '%Data%';
```

Mod_code	Mod_name	Lec_ID
C22	Data Science	L179

Aggregation and Grouping

Aggregate Functions

SQL provides built-in functions that summarise data:

Function	What it does
COUNT(*)	Count all rows
COUNT(col)	Count non-NULL values in a column
SUM(col)	Sum values
AVG(col)	Average
MIN(col)	Minimum value
MAX(col)	Maximum value

```
SELECT COUNT(*) AS total_lecturers FROM Lecturer;
```

```
total_lecturers
```

```
-----
```

```
5
```

GROUP BY: Aggregating by Category

-- How many modules does each lecturer teach?

```
SELECT  Lec_ID,  
        COUNT(*) AS num_modules  
FROM    Module  
WHERE   Lec_ID IS NOT NULL  
GROUP BY Lec_ID  
ORDER BY num_modules DESC;
```

Lec_ID	num_modules
L211	2
L179	2
L555	1

HAVING: Filtering Groups

HAVING is like **WHERE** but applied **after** aggregation:

```
-- Only lecturers teaching more than 1 module
SELECT  Lec_ID,
        COUNT(*) AS num_modules
FROM    Module
WHERE   Lec_ID IS NOT NULL
GROUP BY Lec_ID
HAVING  COUNT(*) > 1;
```

Lec_ID	num_modules
L211	2
L179	2

A common confusion: **WHERE** filters **rows** before grouping; **HAVING** filters **groups** after grouping.

JOINS: Combining Tables

Why JOIN?

Our data is spread across two tables. To answer most interesting questions we need to combine them. This is what JOINS do (we have of course seen this before...).

SQL JOIN	What it keeps
INNER JOIN	Only rows with a match in both tables
LEFT JOIN	All rows from the left table; NULLs for non-matches on the right
RIGHT JOIN	All rows from the right table (SQLite: use reversed LEFT JOIN)
FULL OUTER JOIN	All rows from both tables (not in SQLite, but can be achieved)

The SQL JOIN corresponds directly to the relational algebra **bowtie operator** $\bowtie$.

A Mental Model for JOINS

Before writing any JOIN, ask two questions:

1. **Which table am I primarily interested in?** → That table goes on the LEFT (or is the “base”)
2. **What should happen to rows with no match?** → If you want them kept (with NULLs), use LEFT JOIN; if you want them dropped, use INNER JOIN

“Give me all modules, and attach lecturer info where available”

→ **Module LEFT JOIN Lecturer** (keeps Dissertation with NULL lecturer)

“Give me only modules that have a lecturer”

→ **Module INNER JOIN Lecturer** (drops Dissertation)

Getting the JOIN type wrong is one of the most common sources of silently wrong results in data work. Always check row counts!

INNER JOIN: Only Matching Rows

```
SELECT  m.Mod_code,
        m.Mod_name,
        l.Lec_name,
        l.Lec_dept
FROM    Module    AS m
INNER JOIN Lecturer AS l
ON      m.Lec_ID = l.Lec_ID
ORDER BY l.Lec_name;
```

Mod_code	Mod_name	Lec_name	Lec_dept
-----	-----	-----	-----
C22	Data Science	Amy	Physics
C39	Computer Networks	Amy	Physics
C01	Intro to CS	Abdul	Computer Science
C02	DBMS	Abdul	Computer Science
E15	Econometrics	Sarah	Economics

u01 (NULL Lec_ID) and Marco (no modules) are both **excluded**.

LEFT JOIN: Keep All Modules

```
SELECT  m.Mod_code,
        m.Mod_name,
        l.Lec_name,
        l.Lec_dept
FROM    Module AS m
LEFT JOIN Lecturer AS l
ON      m.Lec_ID = l.Lec_ID
ORDER BY m.Mod_code;
```

Mod_code	Mod_name	Lec_name	Lec_dept
-----	-----	-----	-----
C01	Intro to CS	Abdul	Computer Science
C02	DBMS	Abdul	Computer Science
C22	Data Science	Amy	Physics
C39	Computer Networks	Amy	Physics
E15	Econometrics	Sarah	Economics
U01	Dissertation		

u01 now appears with NULLs for lecturer columns. All modules are retained.

Simulating a RIGHT JOIN in SQLite

SQLite does not support **RIGHT JOIN** – swap the tables in a **LEFT JOIN**:

```
-- All lecturers, including those with no modules
```

```
SELECT  l.Lec_name,
        l.Lec_dept,
        m.Mod_code,
        m.Mod_name
FROM    Lecturer AS l
LEFT JOIN Module AS m
  ON l.Lec_ID = m.Lec_ID
ORDER BY l.Lec_name;
```

Lec_name	Lec_dept	Mod_code	Mod_name
-----	-----	-----	-----
Abdul	Computer Science	C01	Intro to CS
Abdul	Computer Science	C02	DBMS
Amy	Physics	C22	Data Science
Amy	Physics	C39	Computer Networks
Lou	Psychology		
Marco	Social Science		
Sarah	Economics	E15	Econometrics

Lou and Marco now appear with NULLs for module fields.

Understanding JOINS Visually

INNER JOIN — only matched rows

Lecturer	Module
L211 ↔	C01
L211 ↔	C02
L179 ↔	C22
L179 ↔	C39
L555 ↔	E15
L234	(no match) x
R101	(no match) x
	U01 (NULL) x

Returns **5 rows**.

LEFT JOIN — all from the left table

Module LEFT JOIN Lecturer: all 6 modules, U01 gets NULLs → **6 rows**

Lecturer LEFT JOIN Module: all 6 lecturers, Lou and Marco get NULLs → **7 rows** (Abdul and Amy appear twice)

Key insight: The table you want *all* rows from goes on the LEFT.

JOIN with Aggregation

```
-- Modules taught per department (including zero counts)
SELECT  l.Lec_dept      AS Department,
        COUNT(m.Mod_code) AS Modules-Taught
FROM    Lecturer AS l
LEFT JOIN Module  AS m
  ON l.Lec_ID = m.Lec_ID
GROUP BY l.Lec_dept
ORDER BY Modules-Taught DESC;
```

Department	Modules-Taught
-----	-----
Computer Science	2
Physics	2
Economics	1
Psychology	0
Social Science	0

Using `LEFT JOIN` means departments with **no modules** still appear with a count of 0.

Subqueries

A subquery is a `SELECT` nested inside another statement:

```
-- Lecturers who teach at least one module
SELECT * FROM Lecturer
WHERE Lec_ID IN (
    SELECT DISTINCT Lec_ID FROM Module
    WHERE Lec_ID IS NOT NULL
);
```

```
-- Lecturers with NO modules
SELECT * FROM Lecturer
WHERE Lec_ID NOT IN (
    SELECT DISTINCT Lec_ID FROM Module
    WHERE Lec_ID IS NOT NULL
);
```

Lec_ID	Lec_name	Lec_dept
L234	Lou	Psychology
R101	Marco	Social Science

Relational Algebra

The Math Behind SQL

Every SQL query corresponds to an operation in **relational algebra** — the formal mathematical foundation of SQL.

Operation	Symbol	SQL equivalent
Select (filter rows)	σ	<code>WHERE</code>
Project (choose columns)	Π	<code>SELECT col1, col2</code>
Union	$\cup$	<code>UNION</code>
Set difference	$-$	<code>EXCEPT</code>
Cartesian product	$\times$	<code>FROM A, B (no join condition)</code>
Rename	ρ	<code>AS alias</code>
Join	$\bowtie$	<code>JOIN ... ON</code>

You do not need to write relational algebra — but understanding it helps you reason about what SQL is doing.

UNION: Combining Query Results

`UNION` stacks the results of two queries, removing duplicates. `UNION ALL` keeps them.

```
SELECT Lec_dept FROM Lecturer WHERE Lec_dept LIKE 'E%'
UNION
SELECT Lec_dept FROM Lecturer WHERE Lec_dept LIKE 'P%'
ORDER BY Lec_dept;
```

```
Lec_dept
-----
Economics
Physics
Psychology
```

Both queries must return the same number of columns with compatible types.

Data Manipulation Language (DML)

INSERT: Adding Rows

```
INSERT INTO Lecturer VALUES ('L999', 'Elena', 'Economics');
```

```
SELECT * FROM Lecturer WHERE Lec_ID = 'L999';
```

Lec_ID	Lec_name	Lec_dept
-----	-----	-----
L999	Elena	Economics

You can also specify column names explicitly – safer and order-independent:

```
INSERT INTO Lecturer (Lec_ID, Lec_dept, Lec_name)
VALUES ('L998', 'Statistics', 'Omar');
```

UPDATE: Modifying Rows

```
-- Assign the Dissertation to Elena
UPDATE Module
SET    Lec_ID = 'L999'
WHERE  Mod_code = 'U01';

SELECT * FROM Module WHERE Mod_code = 'U01';
Mod_code  Mod_name      Lec_ID
-----
U01       Dissertation    L999
```

 **Always include a WHERE clause with UPDATE.** Without it, every row in the table is modified, and there is no undo.

DELETE: Removing Rows

```
-- Remove Marco - he has no modules  
DELETE FROM Lecturer  
WHERE Lec_ID = 'R101';
```

 **Same warning: DELETE without WHERE empties the entire table.**

```
-- This deletes ALL rows - do not run this by accident  
DELETE FROM Lecturer;
```

Use `SELECT` to preview what you are about to delete:

```
-- Check first  
SELECT * FROM Lecturer WHERE Lec_ID = 'R101';  
-- Then delete  
DELETE FROM Lecturer WHERE Lec_ID = 'R101';
```

Foreign Key Constraints in Action

What happens if we try to delete Abdul, who has modules pointing at him?

By default, SQLite does **not** enforce foreign keys. You must turn it on:

```
PRAGMA foreign_keys = ON;  
  
DELETE FROM Lecturer WHERE Lec_ID = 'L211';  
-- Error: FOREIGN KEY constraint failed
```

Without the PRAGMA, the deletion silently succeeds. This leaves `c01` and `c02` pointing to a lecturer that no longer exists. These are called **orphaned records** and are a serious data integrity problem.

Always enable foreign key enforcement when your schema uses them.

Remote Database Connections

Beyond SQLite: Databases on Servers

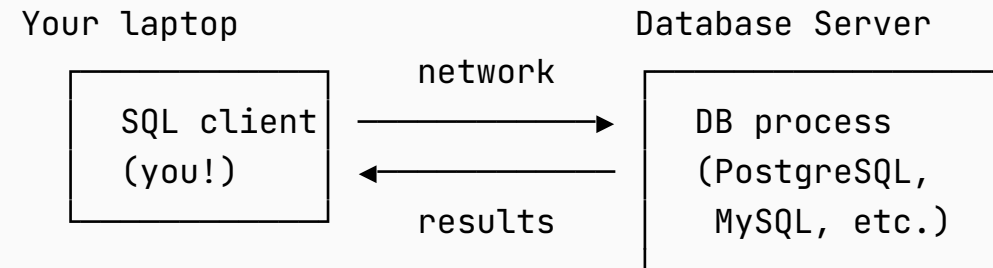
SQLite is a file-based database — the whole thing lives in one file on your local disk. This is great for learning and for many data science tasks.

But in the real world, you will often connect to a database running on a **remote server**:

- A university or company database server
- A cloud-hosted database (AWS RDS, Google Cloud SQL, Azure Database)
- A shared research database you access over a network

The SQL you write is **completely identical**. Only the connection method changes.

The Client-Server Model



- The **server** runs the database engine and holds all the data
- The **client** (you) sends SQL queries and receives results
- Communication happens over TCP/IP on a standard port
- You authenticate with a **username and password**

Your client can be a command-line tool, a GUI app, or Python code – the server does not care which.

Common Database Engines You Will Encounter

Engine	Typical use	CLI client	Default port
SQLite	Local files, learning	<code>sqlite3</code>	(file)
PostgreSQL	Research, analytics, open-source	<code>psql</code>	5432
MySQL / MariaDB	Web apps, shared hosting	<code>mysql</code>	3306
MS SQL Server	Corporate, Windows environments	<code>sqlcmd</code>	1433
Oracle	Large enterprise, finance	<code>sqlplus</code>	1521

PostgreSQL is the most common in academic and research data science. Its SQL dialect is very close to the standard — and to what you have learned today with SQLite.

Connecting with psql and SSH

This is all quite advanced, but worth know it exists in case you come across this. View it as a “set-up cost”

`psql` is PostgreSQL’s command-line client – the direct equivalent of `sqlite3`:

```
# Connect to a remote server
psql -h db.university.ac.uk -p 5432 -U yourname -d researchdb
# You are prompted for a password; prompt becomes: researchdb=>
```

If the database is behind a firewall, tunnel through an SSH gateway:

```
# Step 1: forward the remote DB port to your local machine
ssh -L 5433:db.internal.university.ac.uk:5432 \
yourname@ssh.university.ac.uk -N

# Step 2: in a new terminal, connect via the tunnel
psql -h localhost -p 5433 -U yourname -d researchdb
```

Once connected, the SQL you type is **identical** to what you have been writing today. The `psql` meta-commands differ slightly (`.tables` becomes `\dt`, `.schema` becomes `\d tablename`, `.quit` becomes `\q`) but everything else is the same.

SQL → Python: Bringing It Together 🐼

Why Move to Python at All?

The SQLite3 CLI is excellent for exploration, quick queries, and running scripts. But for **data science workflows**, Python gives you more:

- Fetch data from APIs and web sources (as we will see below)
- Load query results into pandas for analysis and visualisation
- Automate database operations in scripts and pipelines
- Mix SQL data access with statistical modelling

The SQL you have learned is completely unchanged – Python just becomes the interface instead of the terminal.

Connecting from Python

Python's `sqlite3` module is part of the standard library, there is nothing else to install.

You can connect to any `.db` file we created at the CLI (or in-memory):

```
import sqlite3, pandas as pd

# Connect to our teaching database
conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE Lecturer(Lec_ID TEXT PRIMARY KEY, Lec_name TEXT, Lec_dept TEXT);
CREATE TABLE Module(Mod_code TEXT PRIMARY KEY, Mod_name TEXT, Lec_ID TEXT,
    FOREIGN KEY(Lec_ID) REFERENCES Lecturer(Lec_ID));
INSERT INTO Lecturer VALUES('L211','Abdul','CS'),('L555','Sarah','Economics'),
    ('L179','Amy','Physics'),('L234','Lou','Psychology'),('R101','Marco','Social Sci');
INSERT INTO Module VALUES('C01','Intro to CS','L211'),('C02','DBMS','L211'),
    ('C22','Data Science','L179'),('C39','Networks','L179'),
    ('E15','Econometrics','L555'),('U01','Dissertation',NULL);
""")

# pd.read_sql() - the SQL is exactly what we typed at the CLI
result = pd.read_sql("""
SELECT m.Mod_code, m.Mod_name, l.Lec_name, l.Lec_dept
FROM   Module AS m LEFT JOIN Lecturer AS l ON m.Lec_ID = l.Lec_ID
ORDER BY m.Mod_code
""", conn)
print(result)
```

	Mod_code	Mod_name	Lec_name	Lec_dept
0	C01	Intro to CS	Abdul	CS
1	C02	DBMS	Abdul	CS
2	C22	Data Science	Amy	Physics
3	C39	Networks	Amy	Physics

Connecting to PostgreSQL from Python

To connect to a remote PostgreSQL server, use `psycopg2`:

```
pip install psycopg2-binary
```

```
import psycopg2, pandas as pd

# Direct connection
conn = psycopg2.connect(
    host      = "db.university.ac.uk",
    port      = 5432,
    dbname    = "researchdb",
    user      = "yourname",
    password  = "yourpassword"    # better: os.environ["PGPASSWORD"]
)

result = pd.read_sql("SELECT * FROM gdp WHERE year = 2022", conn)
conn.close()
```

If using an SSH tunnel forwarded to `localhost:5433`:

```
conn = psycopg2.connect(
    host="localhost", port=5433, dbname="researchdb", user="yourname"
)
```

SQLAlchemy: One Interface for All Databases

`sqlalchemy` provides a unified connection interface for SQLite, PostgreSQL, MySQL, and more:

```
pip install sqlalchemy psycopg2-binary

from sqlalchemy import create_engine
import pandas as pd

# SQLite (local file)
engine = create_engine("sqlite:///teaching.db")

# PostgreSQL (direct)
engine = create_engine("postgresql://user:pass@db.exeter.ac.uk:5432/bee2041")

# PostgreSQL (via SSH tunnel)
engine = create_engine("postgresql://user:pass@localhost:5433/bee2041")

# The query is identical regardless of which backend you use
df = pd.read_sql("SELECT * FROM Lecturer", engine)
```

This is the recommended approach for production code – your analysis code does not need to change when the “backend” changes.

Best Practice: SQL Does the Heavy Lifting

```
conn2 = sqlite3.connect(":memory:")
conn2.executescript("""
CREATE TABLE Country(iso3 TEXT PRIMARY KEY, name TEXT, region TEXT);
CREATE TABLE GDP(iso3 TEXT, year INTEGER, gdp_bn_usd REAL);
INSERT INTO Country VALUES ('GBR','United Kingdom','Europe'),
    ('FRA','France','Europe'),('DEU','Germany','Europe'),
    ('CHN','China','Asia'),('USA','United States','Americas'),
    ('BRA','Brazil','Americas');
INSERT INTO GDP VALUES ('GBR',2022,3071),('FRA',2022,2782),
    ('DEU',2022,4072),('CHN',2022,17963),('USA',2022,25440),
    ('BRA',2022,1920);
""")

# Use SQL to filter and join - let the DB engine do what it is good at
# Use pandas only for the final analysis step
df = pd.read_sql("""
    SELECT c.region, g.gdp_bn_usd
    FROM    GDP AS g JOIN Country AS c ON g.iso3 = c.iso3
    WHERE   g.year = 2022
""", conn2)

print(df.groupby('region')['gdp_bn_usd'].agg(['sum', 'mean']).round(0))
```

	sum	mean
region		
Americas	27360.0	13680.0
Asia	17963.0	17963.0
Europe	9925.0	3308.0

A Real Example: World Bank Data

Why Real Data?

So far we have worked with a tiny teaching database. Let us use a **real dataset**: world economic indicators from the World Bank Open Data API.

The World Bank publishes thousands of indicators (GDP, population, unemployment, trade, emissions...) for every country, going back to 1960, all freely available via its API. We have also seen this before!

We will:

1. Download data directly from the World Bank API using `requests`
2. Shape it into relational tables and load it into SQLite
3. Query it with SQL – including JOINS and a self-join for growth rates
4. Answer real questions

The Schema: Three Related Tables

```
import sqlite3, wbgapi, pandas as pd

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE Country (
    iso3          TEXT NOT NULL PRIMARY KEY,
    name          TEXT NOT NULL,
    region        TEXT,
    income_grp     TEXT
);
CREATE TABLE Indicator (
    iso3          TEXT NOT NULL,
    year          INTEGER NOT NULL,
    gdp_usd        REAL,
    gdp_pc_usd     REAL,
    unemp_pct      REAL,
    pop            INTEGER,
    PRIMARY KEY (iso3, year),
    FOREIGN KEY (iso3) REFERENCES Country(iso3)
);
""")
print("Schema created.")
```

Schema created.

Fetching from the World Bank API

```
import requests, pandas as pd, sqlite3
from functools import reduce

# The World Bank REST API: no library needed, just requests.
# Endpoint: /v2/country/all/indicator/{code}?format=json&per_page=20000&date=2010:2022
WB = "https://api.worldbank.org/v2/country/all/indicator"

indicators = {
    "NY.GDP.MKTP.CD": "gdp_usd",
    "NY.GDP.PCAP.CD": "gdp_pc_usd",
    "SL.UEM.TOTL.ZS": "unemp_pct",
    "SP.POP.TOTL": "pop",
}

frames = []
for code, colname in indicators.items():
    r = requests.get(f"{WB}/{code}",
                    params={"format": "json", "per_page": 20000, "date": "2010:2022"})
    data = r.json()[1]  # [0] is pagination metadata, [1] is the data
    rows = [{"iso3": d["countryiso3code"],
            "year": int(d["date"]),
            colname: d["value"]}
            for d in data if d["countryiso3code"]]  # drop blanks
    frames.append(pd.DataFrame(rows))
    print(f" {colname}: {len(frames[-1]):,} rows")

raw = reduce(lambda a, b: pd.merge(a, b, on=["iso3", "year"], how="outer"), frames)
# Drop World Bank aggregate rows (iso3 codes that are not real countries)
raw = raw[raw["iso3"] != "XX"]
```

```
gdp_usd: 3,393 rows
gdp_pc_usd: 3,393 rows
unemp_pct: 3,393 rows
pop: 3,393 rows
```

Merged: 3,393 country-year rows, 261 countries

	iso3	year	gdp_usd	gdp_pc_usd	unemp_pct	pop
0	ABW	2010	2.453597e+09	24093.140151	NaN	101838
1	ABW	2011	2.637859e+09	25712.384302	NaN	102591
2	ABW	2012	2.615208e+09	25119.665545	NaN	104110

Loading into SQLite

```
# Country metadata from a separate WB endpoint
r = requests.get("https://api.worldbank.org/v2/country/all",
                 params={"format":"json", "per_page":500})
countries_raw = r.json()[1]
meta = pd.DataFrame([
    "iso3":      c["id"],
    "name":      c["name"],
    "region":    c["region"]["value"],
    "income_grp": c["incomeLevel"]["value"],
} for c in countries_raw if c["region"]["value"] != "Aggregates"])
# Keep only countries that appear in our indicator data
meta = meta[meta["iso3"].isin(raw["iso3"].unique())].copy()

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE Country (
    iso3 TEXT NOT NULL PRIMARY KEY, name TEXT NOT NULL,
    region TEXT, income_grp TEXT
);
CREATE TABLE Indicator (
    iso3 TEXT NOT NULL, year INTEGER NOT NULL,
    gdp_usd REAL, gdp_pc_usd REAL, unemp_pct REAL, pop REAL,
    PRIMARY KEY (iso3, year),
    FOREIGN KEY (iso3) REFERENCES Country(iso3)
);
""")
meta.to_sql("Country", conn, if_exists="replace", index=False)
raw.to_sql("Indicator", conn, if_exists="replace", index=False)
```

```
countries
0      217
obs
0  3321
```

Now we have a **real, relational database** — the same two-table structure we have been building all day, populated with live World Bank data.

Querying: GDP Rankings

```
result = pd.read_sql("""
    SELECT c.name,
           c.region,
           c.income_grp,
           ROUND(i.gdp_usd / 1e12, 2) AS gdp_trillion_usd
    FROM   Indicator AS i
    JOIN   Country   AS c ON i.iso3 = c.iso3
    WHERE  i.year = 2022
    AND    i.gdp_usd IS NOT NULL
    ORDER BY i.gdp_usd DESC
    LIMIT  12
""", conn)
print(result.to_string(index=False))
```

name	region	income_grp	gdp_trillion_usd
United States	North America	High income	25.60
China	East Asia & Pacific	Upper middle income	18.32
Japan	East Asia & Pacific	High income	4.26
Germany	Europe & Central Asia	High income	4.20
India	South Asia	Lower middle income	3.35
United Kingdom	Europe & Central Asia	High income	3.18
France	Europe & Central Asia	High income	2.79
Russian Federation	Europe & Central Asia	High income	2.29
Canada	North America	High income	2.19
Italy	Europe & Central Asia	High income	2.10
Brazil	Latin America & Caribbean	Upper middle income	1.95
Korea, Rep.	East Asia & Pacific	High income	1.80

Querying: GDP Per Capita by Region

```
result = pd.read_sql("""
    SELECT      c.region,
                COUNT(DISTINCT i.iso3)           AS countries,
                ROUND(AVG(i.gdp_pc_usd))         AS avg_gdp_pc,
                ROUND(MIN(i.gdp_pc_usd))         AS min_gdp_pc,
                ROUND(MAX(i.gdp_pc_usd))         AS max_gdp_pc
    FROM        Indicator AS i
    JOIN        Country   AS c ON i.iso3 = c.iso3
    WHERE       i.year = 2022
    AND         i.gdp_pc_usd IS NOT NULL
    GROUP BY    c.region
    ORDER BY    avg_gdp_pc DESC
""", conn)
print(result.to_string(index=False))
```

	region	countries	avg_gdp_pc	min_gdp_pc	max_gdp_pc
	North America	3	84843.0	56257.0	121614.0
	Europe & Central Asia	57	40174.0	1052.0	226052.0
Middle East, North Africa, Afghanistan & Pakistan		22	18804.0	357.0	88701.0
	East Asia & Pacific	36	18273.0	1158.0	90299.0
	Latin America & Caribbean	39	16870.0	1761.0	93031.0
	South Asia	6	4215.0	1386.0	11786.0
	Sub-Saharan Africa	46	2601.0	303.0	16837.0

Aggregation + JOIN on real data – and notice the enormous variation within regions (min vs max GDP per capita).

Self-Join: GDP Growth Rates

```
result = pd.read_sql("""
    SELECT c.name,
           c.region,
           ROUND(a.gdp_usd / 1e12, 2)           AS gdp_2012,
           ROUND(b.gdp_usd / 1e12, 2)           AS gdp_2022,
           ROUND(100.0*(b.gdp_usd - a.gdp_usd)
                 / a.gdp_usd, 1)                AS growth_10yr_pct
    FROM   Indicator AS a
    JOIN   Indicator AS b
        ON  a.iso3 = b.iso3 AND a.year = 2012 AND b.year = 2022
    JOIN   Country   AS c ON a.iso3 = c.iso3
    WHERE  a.gdp_usd IS NOT NULL AND b.gdp_usd IS NOT NULL
    ORDER BY growth_10yr_pct DESC
    LIMIT  10
""", conn)
print(result.to_string(index=False))
```

name	region	gdp_2012	gdp_2022	growth_10yr_pct
Guyana	Latin America & Caribbean	0.00	0.01	262.2
Bangladesh	South Asia	0.13	0.46	245.2
Ethiopia	Sub-Saharan Africa	0.04	0.12	184.3
Timor-Leste	East Asia & Pacific	0.00	0.00	176.5
Djibouti	Middle East, North Africa, Afghanistan & Pakistan	0.00	0.00	163.2
Guinea	Sub-Saharan Africa	0.01	0.02	160.7
Ireland	Europe & Central Asia	0.23	0.55	141.6
Sao Tome and Principe	Sub-Saharan Africa	0.00	0.00	139.5
Zimbabwe	Sub-Saharan Africa	0.02	0.04	138.0
Congo, Dem. Rep.	Sub-Saharan Africa	0.03	0.07	134.5

Unemployment vs Income Group

```
result = pd.read_sql("""
    SELECT    c.income_grp,
              COUNT(*)                               AS n,
              ROUND(AVG(i.unemp_pct), 1)              AS avg_unemp,
              ROUND(MIN(i.unemp_pct), 1)              AS min_unemp,
              ROUND(MAX(i.unemp_pct), 1)              AS max_unemp
    FROM      Indicator AS i
    JOIN      Country   AS c ON i.iso3 = c.iso3
    WHERE     i.year = 2022
    AND       i.unemp_pct IS NOT NULL
    GROUP BY  c.income_grp
    ORDER BY  avg_unemp DESC
    """, conn)
print(result.to_string(index=False))
```

	income_grp	n	avg_unemp	min_unemp	max_unemp
Upper middle income	48	8.9	0.9	33.3	
Lower middle income	48	8.1	0.2	36.5	
Low income	25	6.5	0.4	19.1	
High income	63	5.6	0.1	13.0	
Not classified	2	4.5	3.4	5.6	

This is the kind of cross-country query economists do all the time — and it took about four lines of SQL on a live, real dataset.

Missing Data: A Real SQL Problem

Real data always has NULLs. Let's look at data completeness:

```
result = pd.read_sql("""
    SELECT year,
           COUNT(*)                               AS total_countries,
           SUM(CASE WHEN gdp_usd IS NOT NULL THEN 1 ELSE 0 END) AS gdp_available,
           SUM(CASE WHEN unemp_pct IS NOT NULL THEN 1 ELSE 0 END) AS unemp_available
    FROM   Indicator
    GROUP BY year
    ORDER BY year DESC
    LIMIT  10
    """, conn)
print(result.to_string(index=False))
```

year	total_countries	gdp_available	unemp_available
2022	261	253	230
2021	261	254	231
2020	261	254	231
2019	261	255	231
2018	261	255	231
2017	261	255	231
2016	261	255	231
2015	261	256	231
2014	261	257	231
2013	261	256	231

Notice how data availability varies by year and indicator — something to check *before* running regressions. SQL lets you audit this directly.

Saving the Database to a File

To work with this database from the SQLite3 CLI, save it to a file:

```
# Save the in-memory database to a real file
import shutil

conn_file = sqlite3.connect("worldbank.db")
conn.backup(conn_file)
conn_file.close()
print("Saved to worldbank.db – open it with:  sqlite3 worldbank.db")
```

Saved to worldbank.db – open it with: sqlite3 worldbank.db

```
# Now explore interactively at the command line!
sqlite3 worldbank.db
.headers on
.mode column
.tables
SELECT name, region FROM Country WHERE income_grp = 'High income' LIMIT 10;
```

This is the full workflow: Python fetches and shapes real data → SQL stores and queries it → CLI for exploration.

Views and Indexes

Views: Saved Queries

A **view** is a named query stored in the database – it behaves like a table but recomputes its result each time it is queried:

```
CREATE VIEW vw_module_lecturer AS
SELECT m.Mod_code, m.Mod_name,
       l.Lec_name, l.Lec_dept
FROM   Module   AS m
JOIN   Lecturer AS l ON m.Lec_ID = l.Lec_ID;

-- Query it like a table
SELECT * FROM vw_module_lecturer WHERE Lec_dept = 'Physics';
```

Mod_code	Mod_name	Lec_name	Lec_dept
C22	Data Science	Amy	Physics
C39	Computer Networks	Amy	Physics

Views are useful for: sharing consistent query logic, hiding complexity, and security (expose the view but not the underlying tables).

Indexes: Making Queries Fast

For large databases, queries on unindexed columns require a **full table scan**: every row is checked.

```
-- On a 10-million-row table, this is slow without an index
SELECT * FROM GDP WHERE iso3 = 'GBR';

-- Create an index
CREATE INDEX idx_gdp_iso3 ON GDP(iso3);

-- Now the same query jumps straight to the right rows
SELECT * FROM GDP WHERE iso3 = 'GBR';
```

Rules of thumb:

- Primary keys are indexed automatically
- Index columns you filter on frequently (`WHERE`, `JOIN ON`)
- Indexes speed up reads but slow down writes (the index must be maintained)
- Do not over-index: each index costs storage and has performance implications

NoSQL: A Brief Overview

What About NoSQL?

You may encounter **NoSQL** databases in data science work – especially with web data:

- RDBMS require a pre-defined schema and structured data
- What if data is semi-structured or schema-free? (JSON, logs, tweets, etc.)
- What if you need to scale across hundreds of machines?

NoSQL (“Not only SQL”) offers different trade-offs:

Type	Example	Use case
Document	MongoDB	JSON-style documents
Key-Value	Redis	Caching, sessions
Column-family	Cassandra	Time series, analytics
Graph	Neo4j	Social networks, recommendations

When to Use What?

Use SQL/RDBMS when:

- Data is structured and relational
- Data integrity is critical
- You need complex queries and joins
- Team knows SQL (everyone does!)
- Examples: financial data, survey data, administrative records

Consider NoSQL when:

- Data is unstructured or highly variable (web scraping, social media)
- You need massive horizontal scalability
- Write performance is critical at huge scale
- Schema changes frequently

For most economics data science work: **SQL is the right choice.**

If you want to explore NoSQL: look up **MongoDB** – it stores data as JSON documents and is widely used.

Summary

A SQL Query Checklist

Before writing a complex query, ask yourself:

1. **Which tables** do I need? → FROM and JOIN
2. **Which rows** do I want? → WHERE
3. **Which columns** do I need? → SELECT
4. **Do I need aggregation?** → GROUP BY and HAVING
5. **How should results be ordered?** → ORDER BY
6. **Are there NULL values** I need to handle? → IS NULL, COALESCE

Common Mistakes to Avoid

- ✗ `UPDATE` or `DELETE` without a `WHERE` clause - affects every row, permanently
- ✗ `WHERE col = NULL` - always use `IS NULL`
- ✗ Confusing `WHERE` (before aggregation) and `HAVING` (after aggregation)
- ✗ Not enabling `PRAGMA foreign_keys = ON` in SQLite when your schema uses them
- ✗ Running large unfiltered queries on shared servers - use `LIMIT` to explore first
- ✗ Hardcoding passwords in scripts - use environment variables or `.pgpass`
- ✗ Forgetting that `sqlite3` (the CLI tool) and `SQLite3` (Python module) need their connections managed separately

Essential Reference

CLI: `sqlite3 file.db`, `.headers on`, `.mode column`, `.tables`, `.schema`, `.read`, `.quit`

SQL: `SELECT`, `FROM`, `WHERE`, `ORDER BY`, `LIMIT`, `GROUP BY`, `HAVING`, `INNER/LEFT JOIN`, `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`, `CREATE VIEW`

Remote: `psql -h host -U user -d db`, SSH tunnelling, connection URIs, `GRANT`/`REVOKE`

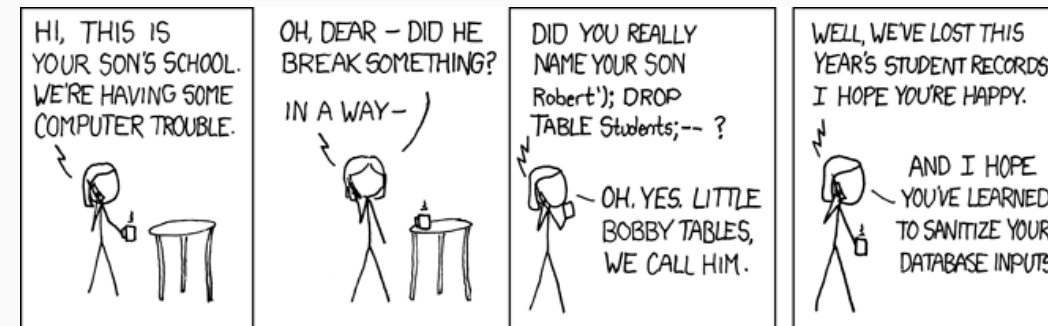
Python: `sqlite3.connect()`, `pd.read_sql()`, `df.to_sql()`, `psycopg2`, `sqlalchemy`

Practical Exercise

At the command line:

1. Create `teaching.db` from the provided SQL script
2. Extend the schema with a `student` table and an `Enrolment` junction table
3. Insert some students and enrolments
4. Query: which students take which modules, which lecturers teach the most students
5. Try deleting a lecturer who has enrolments - with and without `PRAGMA foreign_keys = ON`

Extension: Connect to the same `.db` file from Python using `pd.read_sql()` and reproduce your results.



xkcd.com/327

Resources for Further Learning

Documentation

- [SQLite official docs](#)
- [PostgreSQL docs](#)
- [Python sqlite3 module](#)
- [SQLAlchemy docs](#)

Books

- *SQL in a Nutshell* a compact, practical reference
- *SQL and Relational Theory* by C. J. Date, more rigorous treatment

Practice

- [SQLZoo](#) — interactive exercises, browser-based
- [Mode Analytics SQL Tutorial](#)
- Real data: World Bank, ONS, FRED — download as CSV, load into SQLite, query away

Remember: the best way to learn SQL is to break things and fix them. Start simple, always check your results and keep getting comfortable at the command line 🚀